CITS5508 Practice Exam — Chapter 4 — ANSWER KEY

Question 1.

(a) *(4 marks)* Prefer gradient descent when (any two): - There are **very many features** (e.g. hundreds of thousands) — the Normal Equation inverts an (n+1)×(n+1) matrix at **$O(n^{2.4}–n^3)$**, which is prohibitive, while GD scales well with n. - The **training set is too large to fit in memory** — stochastic/mini-batch GD can learn incrementally / out-of-core. - (Also acceptable: GD generalises to models with **no closed-form solution**, e.g. logistic regression.)

(b) *(4 marks)* All compute the gradient of the cost and step downhill, differing in how much data per step: - **Batch GD** — uses the **whole** training set each step (stable, but slow on large m). - **Stochastic GD (SGD)** — uses **one random instance** per step. - **Mini-batch GD** — uses a **small random batch** per step. - Advantage of SGD over batch: **much faster per step / handles huge or out-of-core data**, and its randomness can help **escape local minima**.

(c) *(4 marks)* - η **too small** → tiny steps → **very slow** convergence (many iterations). - η **too large** → overshoots, may **diverge** (cost grows, bounces across the valley). - **Feature scaling** matters because with unscaled features the cost surface is an **elongated bowl**, so GD **zig-zags** and converges slowly; scaling makes it roughly spherical so GD heads straight to the minimum.

Question 2.

(a) *(6 marks)* A learning curve plots the model's **training error** and **validation error** as a function of the **training-set size** (or training iterations). - **Underfitting:** both curves reach a **plateau**, **close together** and **fairly high** (training error rises from ~0 to a high plateau; validation error falls to meet it). Adding more data does not help. - **Overfitting:** training error stays **low**, with a large **gap** to a much higher validation error; the gap **shrinks** as more data is added. *(3 marks per case: correct shape + correct interpretation.)*

(b) *(4 marks)* - **Bias** — error from **wrong assumptions** (e.g. assuming linear when quadratic); high bias → underfitting. - **Variance** — error from **excessive sensitivity** to small training-data variations; high variance → overfitting. - **Irreducible error** — error from **noise in the data** itself; only reduced by cleaning the data. - Increasing model complexity **reduces bias** (and increases variance).

(c) *(2 marks)* Training ≈ validation error, both **high** → the model **underfits** → **high bias**. α controls regularisation, and more α = more bias, so to reduce bias you should **decrease α** (a less constrained model).

Question 3.

(a) *(3 marks)* Both add a penalty to the MSE. **Ridge** adds α times the **squared ℓ_2 norm** of the weights ($\alpha \cdot \Sigma \theta_i^2$); **lasso** adds a term proportional to the **ℓ_1 norm** ($2\alpha \cdot \Sigma |\theta_i|$). (The bias term is not penalised.)

(b) *(4 marks)* The ℓ_1 penalty has a **constant gradient (±1)** regardless of a weight's magnitude, so it keeps pushing small weights all the way **to exactly zero** (and they stay there) → automatic feature selection / **sparse** model. The ℓ_2 penalty's gradient **shrinks as the weight approaches zero**, so ridge slows down near zero and only **shrinks** weights without eliminating them.

(c) *(3 marks)* 1. **Ridge** — a good default with light regularisation, no reason to drop features. 2. **Lasso** (or elastic net) — zeros out useless features (sparse model). 3. **Elastic net** — preferred over lasso when features **outnumber instances** or are **strongly correlated** (lasso behaves erratically there).

(d) *(2 marks)* **Early stopping:** during iterative training (e.g. GD), monitor the **validation error** and **stop as soon as it reaches its minimum** (it then starts rising = overfitting). Roll back to the best-recorded parameters. Simple, cheap regularisation.

Question 4.

(a) *(4 marks)* 50% chance ⇒ $\hat{p} = 0.5$ ⇒ logit $t = 0$: $-4 + 0.05 \cdot h + 0.8 \cdot (3.0) = 0$ $0.05 \cdot h + 2.4 - 4 = 0 \rightarrow 0.05 \cdot h = 1.6 \rightarrow$ **$h = 32$ hours**.

(b) *(3 marks)* $t = -4 + 0.05 \cdot (40) + 0.8 \cdot (3.0) = -4 + 2 + 2.4 =$ **0.4**. $\hat{p} = \sigma(0.4) = 1/(1 + e^{-0.4}) = 1/(1 + 0.6703) \approx 1/1.6703 \approx$ **0.599 (≈ 60%)**.

(c) *(3 marks)* The sigmoid $\sigma(t) = 0.5$ exactly when **$t = 0$** (and σ is increasing). Since $\hat{p} = \sigma(\text{logit})$, $\hat{p} = 0.5 \Leftrightarrow \text{logit} = \theta^T x = 0$. The set of points where $\theta^T x = 0$ is a **hyperplane (a line in 2-D)** → logistic regression has a **linear decision boundary**.

(d) *(2 marks)* Use **two logistic regression** classifiers. The two axes (indoor/outdoor, day/night) are **independent** (an instance has one label on each) — this is a **multilabel** problem. **Softmax** is for a **single set of mutually exclusive** classes, so it is not appropriate here.

Question 5.

(a) *(6 marks)*

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error

grid = GridSearchCV(
    Ridge(),
    {"alpha": [0.01, 0.1, 1.0, 10.0, 100.0]},   # five values
    cv=3,
    scoring="neg_mean_squared_error")
grid.fit(X_train_scaled, y_train)               # refit=True → best model on full
↪  train set

print("Best alpha:", grid.best_params_["alpha"])
y_pred = grid.predict(X_test_scaled)
print("Test MSE:", mean_squared_error(y_test, y_pred))
```

(Award for: Ridge + GridSearchCV with 5 alphas, cv=3, fit on scaled train, printing best α, computing test MSE. `GridSearchCV` refits the best estimator on the whole training set by default.)

(b) *(2 marks)* `C` is the **inverse** of regularisation strength: **higher C ⇒ less regularisation** (lower C ⇒ stronger regularisation).

(c) *(4 marks)* Degree-3 features for x_1, x_2 (no bias term): x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, $x_1^2x_2$, $x_1x_2^2$, x_2^3 *(degree-1: x_1, x_2; degree-2: x_1^2, x_1x_2, x_2^2; degree-3: x_1^3, $x_1^2x_2$, $x_1x_2^2$, x_2^3 — 9 features.)*